

Memory Caching — 한국어의역본

이 문서는 arXiv:2602.24281 「Memory Caching: RNNs with Growing Memory」 (Behrouz, Li, Deng, Zhong, Razaviyayn, Mirrokni (Google / USC)) 의 한국어 의역본입니다. 스터디용이며 공식 번역이 아닙니다. 수식 · 수치 · 그림은 원문 기준이고, 그림은 원저자의 것입니다. 자연스러운 이해를 위해 문장은 의역했습니다.

초록 (Abstract)

Transformer 는 최근 시퀀스 모델링 발전 대부분의 사실상 표준 backbone 으로 자리 잡았는데, 그 주된 이유는 **문맥 길이에 따라 함께 자라는 메모리 용량**이다. 이는 retrieval 과제에는 그럴듯하지만 이차 (quadratic) 복잡도를 유발하며, 그래서 실용적인 subquadratic recurrent 대안을 찾는 연구들을 촉발해 왔다. 이런 recurrent 아키텍처들은 다양한 도메인에서 유망한 초기 결과를 보였음에도 **recall 집약적 과제에서는 Transformer 에 뒤처지는데**, 그 원인은 흔히 **고정 크기 메모리**에 있다고 지목된다. 본 논문에서 우리는 **Memory Caching(MC)** 을 제안한다. recurrent 모델의 메모리 상태 (즉 hidden state) 의 **체크포인트를 캐시**하는, 단순하지만 효과적인 기법이다. MC 는 RNN 의 유효 메모리 용량이 시퀀스 길이에 따라 자라도록 하며, RNN 의 고정메모리 ($O(L)$ 복잡도) 와 Transformer 의 자라는 메모리 ($O(L^2)$ 복잡도) **사이를 보간하는 유연한 트레이드오프**를 제공한다. 우리는 gated aggregation 과 sparse selective 메커니즘을 포함한 **네 가지 MC 변형**을 제안하고, linear 및 deep memory module 각각에서의 함의를 논한다. 언어 모델링과 긴 문맥 이해 과제 실험은 MC 가 recurrent 모델의 성능을 끌어올림을 보인다. in-context recall 과제에서는 Transformer 가 최고정확도를 내지만, MC 변형들은 경쟁력 있는 성능으로 Transformer 와의 격차를 좁히며 최신 recurrent 모델들을 앞선다.

1 서론 (Introduction)

Transformer 의 힘의 큰 부분은 모든 과거 token 을 KV 캐시에 그대로 보관하는 **자라는 메모리**에서 온다 — 어떤 과거든 직접 접근할 수 있으니 recall 이 강하다. 대가는 이차 비용이다. 반대편의 recurrent 모델 (현대적 RNN: linear attention 계열, SSM, deep memory 계열) 은 **고정 크기** 상태로 시퀀스를 압축해 선형 비용을 얻지만, 시퀀스가 길어지면 상태가 넘쳐 (과적재) recall 이 무너진다. 이 긴장 — 자라는 메모리의 recall vs 고정메모리의 효율 — 이 본 논문의 출발점이다.

우리의 제안은 중간 지대다: recurrent 메모리의 **중간상태 (체크포인트) 를 캐시**해서, 이후 token 들이 과거의 관련 구간에 **직접 접근**할 수 있게 하는 것이다 (그림 1). 시퀀스를 세그먼트로 나누고 각 세그먼트가 끝날 때의 메모리 상태를 저장해 두면, 유효 메모리는 세그먼트 수 N 에 비례해 자라고, 복잡도는 $O(L)$ 과 $O(L^2)$ 사이의 $O(NL)$ 로 보간된다.

기여는 다음과 같다.

- **Memory Caching(MC)**: 임의의 recurrent update rule 에 적용가능한, 메모리 상태 체크포인트 캐싱 일반 기법.
- **네 가지 집계 (aggregation) 전략**: (i, ii) **(Gated) Residual Memory** — 캐시들의 (게이트된) 잔차 합산; (iii) **Memory Soup** — weight souping 에서 영감을 받아 캐시된 메모리 모듈의 **파라미터 자체**를 평균 (비선형 메모리에서 고유한 의미를 가짐); (iv) **Sparse Selective Caching(SSC)** — Mixture-of-Experts 스타일 router 로 문맥상 관련 있는 캐시만 골라 효율적으로 집계.
- linear/deep memory 각각에서의 함의 분석과, Titans-linear attention 계열에 대한 개념 증명 실험.

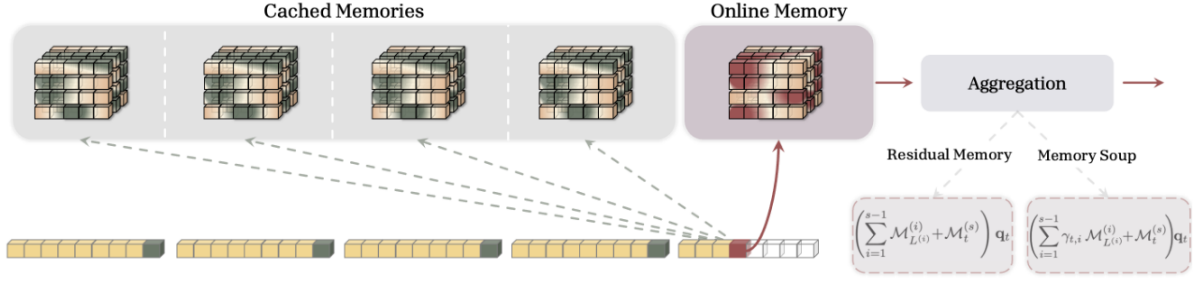


Figure 1: The Overall Memory Caching Method. Each token attends to its online memory as well as a set of cached memories from the past.

그림 1: Memory Caching 전체 개요 — 각 token 은 자신의 온라인메모리와 함께 과거 세그먼트들의 캐시된 메모리에 접근한다. (출처: 원저자, arXiv:2602.24281 Fig.1)

2 예비지식과 배경 (Preliminaries and Background)

attention 과 그 linear 변형들을 복기한다. linear attention(Katharopoulos et al., 2020) 은 softmax 를 kernel feature map 으로 대체해, 인과 (causal) 형태에서는 외적 누적 상태를 가진 RNN 이 된다:

$$M_t = M_{t-1} + v_t k_t^\top, \quad y_t = M_t q_t.$$

이 관점에서 메모리 갱신은 **연상 기억 (associative memory)** 을 학습시키는 **online 최적화 과정**으로 볼 수 있고 (파라미터적 in-context learning, nested learning 관점), 시퀀스의 token 들이 그 학습의 훈련샘플이 된다. 캐시된 상태는 이 **최적화 과정의 체크포인트**로서, 과거를 회수 (retrieve) 하는 능력을 강화한다.

3 Memory Caching 을 가진 RNN (RNNs with Memory Caching)

RNN 은 고정 크기 메모리로 입력을 압축하므로, 시퀀스가 길어지면 메모리가 넘치고 성능이 떨어진다. 반대로 attention 은 모든 과거 token 을 캐시해 메모리가 자라는 대신 비용이 이차다. **MC 는 중간 메모리 상태를 캐시해**, 모델의 메모리가 임의 스케일로 자랄 수 있는 중간 지대를 제공한다.

시퀀스 $x \in \mathbb{R}^{L \times d_{in}}$ 을 세그먼트 $S^{(1)}, \dots, S^{(N)}$ (길이 $L^{(1)}, \dots, L^{(N)}$) 으로 나누고, 각 세그먼트를 메모리 $M^{(1)}, \dots, M^{(N)}$ 으로 압축한다. s 번째 세그먼트의 메모리 갱신은:

$$\begin{aligned} k_t &= x_t W_k, \quad v_t = x_t W_v, \quad q_t = x_t W_q, \\ M_t^{(s)} &= f\left(M_{t-1}^{(s)}; k_t, v_t\right), \quad 1 \leq t \leq L^{(s)}, \end{aligned} \quad (4)$$

여기서 $f(\cdot)$ 는 학습 update rule 이다 (예: linear attention 이면 $f = M_{t-1} + v_t k_t^\top$). 갱신 후 각 세그먼트의 **마지막 상태를 캐시**한다: $\{M_{L^{(s)}}^{(s)}\}_{s=1}^T$. 표준 RNN 은 현재 메모리만으로 출력을 계산하지만 ($y_t = M_t(q_t)$), MC 는 **캐시된 모든 메모리와 현재 (온라인) 메모리를 함께 사용**한다. 임의의 집계 함수 Agg 에 대해:

$$y_t = \text{Agg}\left(\{M_{L^{(1)}}^{(1)}(\cdot), \dots, M_{L^{(s-1)}}^{(s-1)}(\cdot)\}; M_t^{(s)}(\cdot); q_t\right). \quad (5)$$

$M_{L^{(i)}}^{(i)}(q_t)$ 는 “세그먼트 i 에서 q_t 에 대응하는 정보” 를 준다. 이하에서 Agg 의 효과적인 선택지들을 제시한다.

3.1 Residual Memory 와 Gated Residual Memory (GRM)

가장 단순한 Agg 는 **합산** — 메모리 상태들을 가로지르는 잔차 연결이다:

$$y_t = \underbrace{M_t^{(s)}(q_t)}_{\text{온라인 메모리}} + \underbrace{\sum_{i=1}^{s-1} M_{L(i)}^{(i)}(q_t)}_{\text{캐시된 메모리들}}. \quad (7)$$

결정적 변화는 출력 계산 방식이다: 회수 시 현재 메모리뿐 아니라 캐시들에 대해서도 forward pass 를 수행한다.

Gated Residual Memory (GRM). 메모리가 엄밀히 선형 (행렬) 이면 식 (7) 은 캐시들을 미리 합쳐둘 수 있어 수학적으로는 고정 크기 메모리로 **붕괴 (collapse)** 한다 (식 13 참조). 그럼에도 실험에서는 이 단순한 형태조차 성능을 높이는데, 잔차 메모리가 먼 과거 접근을 강화하는 일종의 **retention 연산자**로 작동하기 때문이다. 잔차 방식의 또 다른 한계는 모든 캐시를 **동등하게** 취급해 query 와의 관련성을 무시한다는 점이다. 선택적 회수를 위해 입력 의존 게이트 $0 \leq \gamma_t^{(i)} \leq 1$ 을 도입한다:

$$y_t = \gamma_t^{(s)} M_t^{(s)}(q_t) + \sum_{i=1}^{s-1} \gamma_t^{(i)} M_{L(i)}^{(i)}(q_t). \quad (9)$$

입력 의존 파라미터 때문에 이 형태는 미리 계산해 둘 수도, 다음 token 에 재사용할 수도 없다 — 따라서 **선형 메모리에서도 고정 크기로 붕괴하지 않으며**, 매 token 재계산과 상태 캐싱이 필요하다. $\gamma_t^{(i)}$ 를 입력의 선형 사영만으로 두면 위치 기반 필터에 그치므로, x_t 와 세그먼트 $S^{(i)}$ 양쪽의 문맥을 반영하도록 connector 파라미터 u_t 를 도입해 유사도로 정의한다:

$$\gamma_t^{(i)} = \left\langle u_t, \text{MeanPooling}(S^{(i)}) \right\rangle, \quad u_t = x_t W_u. \quad (10)$$

MeanPooling 은 세그먼트 문맥의 단순 대표 (전체 token 평균) 이며 다른 pooling 으로 대체 가능하다. 실전에서는 γ 를 softmax 로 정규화한다. 대안으로 $u_t = q_t$ 도 쓸 수 있다. γ 가 상수면 GRM 은 residual memory 와 같아진다.

예시. $f(M_{t-1}^{(s)}; k_t, v_t) = M_{t-1}^{(s)} - \nabla \langle M_{t-1}^{(s)}(k_t), v_t \rangle$ 로 두면 (메모리 M 은 임의의 feedforward 층 — MLP·gated MLP), 이는 **Deep Linear Attention (DLA)** (Behrouz et al., 2025a) 과 같고, 메모리가 행렬이면 linear attention 과 같다. DLA 에 residual caching 을 적용하면:

$$M_t^{(s)} = M_{t-1}^{(s)} - \nabla \langle M_{t-1}^{(s)}(k_t), v_t \rangle, \quad y_t = M_t^{(s)}(q_t) + \sum_{i=1}^{s-1} M_{L(i)}^{(i)}(q_t). \quad (11)$$

선형 (행렬) 메모리라면 식 (11) 은 다음처럼 단순해진다:

$$y_t = \left(M_t^{(s)} + \sum_{i=1}^{s-1} M_{L^{(i)}}^{(i)} \right) q_t. \quad (13)$$

메모리 복잡도. 갱신 과정은 변하지 않아 $O(L)$ 이지만, 회수는 캐시전체에 대한 forward 가 필요해 token 당 $O(N)$ — 전체 $O(NL)$, $1 \leq N \leq L$. $N=1$ 이면 캐시가 없는 단순 recurrent 모델, $N=L$ 이면 모든 과거 token 의 상태가 캐시되어 attention 의 직관 — 모든 과거에의 직접 접근 — 과거의 일치한다.

3.2 Memory Soup

recurrence 를 메모리 상태가 체크포인트인 **메타 학습 과정**으로보고, weight souping(Wortsman et al., 2022) 에서 영감을 받은 변형을제안한다. 핵심은 캐시된 메모리 상태 (파라미터) 들을 **하나의 데이터존 메모리로 결합해 회수에 쓰는 것**이다. 캐시 $M_{L^{(i)}}^{(i)}$ 의파라미터를 $\theta_{M^{(i)}} = \{W_1^{(i)}, \dots, W_c^{(i)}\}$ 라하면 (아키텍처는 동일하므로 파라미터 수 c 도 동일):

$$y_t = M_t^*(q_t), \quad \theta_{M_t^*} := \left\{ \sum_{i=1}^s \gamma_t^{(i)} W_1^{(i)}, \dots, \sum_{i=1}^s \gamma_t^{(i)} W_c^{(i)} \right\}. \quad (14-15)$$

즉 각 token 이 자기만의 회수용 메모리를 그때그때 지어쓴다 (γ 는 식 10 과 동일). 메모리가 선형이면 Soup 은 GRM 과수학적으로 동치다 — weight 를 souping 한 뒤 query 를 적용하는 것과, 각메모리에 query 를 적용한 뒤 출력을 앙상블하는 것이 선형성 때문에 같기때문이다. **차이**는 deep/비선형메모리 (DLA·Titans) 에서 결정적이 된다: 동치가 깨지고, Soup 은 파라미터 자체를 보간해 그 timestep 전용의 비선형 회수함수를 만들어 낸다.

3.3 Sparse Selective Caching (SSC)

앞의 변형들은 모든 캐시에 접근하므로 초장문에서는 부담이 크다. **SSC** 는 각 token 이 캐시의 부분집합만 문맥적으로 선택하게한다. MoE(Shazeer et al., 2017) 에서 영감을 받아, token 과 각 세그먼트문맥의 유사도로 router 가 고른다. $\text{MeanPooling}(S^{(i)}) = \sum_{j \in S^{(i)}} k_j$ 로 두고관련도 점수를

$$r_t^{(i)} = \left\langle u_t, \text{MeanPooling}(S^{(i)}) \right\rangle, \quad u_t = x_t W_u \quad (16)$$

로 정의한 뒤, 상위 k 개 $R_t = \arg \text{Top-k}(\{r_t^{(i)}\})$ 와 온라인 메모리만으로회수한다:

$$y_t = \gamma_t^{(s)} M_t^{(s)}(q_t) + \sum_{i \in R_t} \gamma_t^{(i)} M_{L^{(i)}}^{(i)}(q_t). \quad (17)$$

MeanPooling 은 사전 계산 가능해 관련도 계산과 Top-k 선택은 병렬화가 쉽고, 캐시 상태들을 accelerator 에 상주시킬 필요 없이 “선택된” 메모리만 로드하면 되므로 훈련 · 추론모두에서 메모리 소비를 줄인다.

유효 메모리 해석. SSC 는 희소하게 활성화되는 하나의 통합메모리로 볼 수 있다 (그림 2 오른쪽): 각 token 이 쓰기 (저장) 에는파라미터의 한 부분집합을, 회수에는 더 큰 부분집합을 활성화하

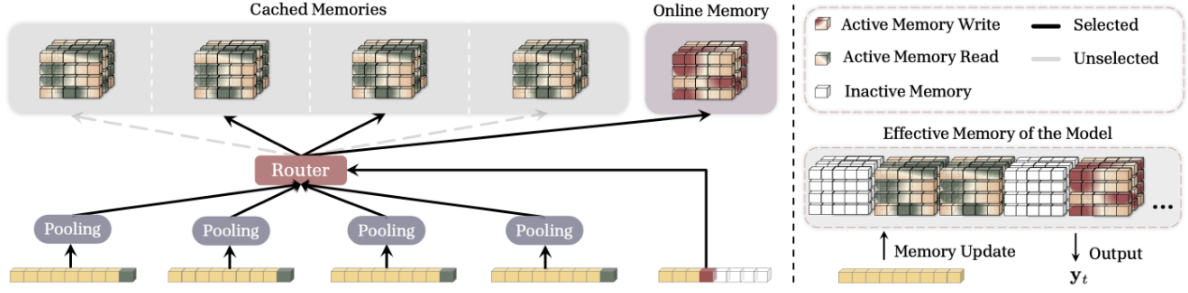


Figure 2: Sparse Selective Caching (SSC) of Memories. A router measures the contextual similarity of each token to its past segments and chooses a subset of past cached memory for better efficiency.

그림 2: SSC — router 가 token 과 과거 세그먼트들의 문맥 유사도를 재고 캐시의 부분집합만 고른다. (출처: 원저자, arXiv:2602.24281 Fig.2)

는, **자라는 메모리** 모델이다. 이 형태는 (1) 과거 메모리와 간섭 없이 저장하고, (2) 효율적 · 적응적으로 회수하게 해 준다. 세그먼트 크기가 통합 메모리에서 함께 활성화되는 블록의 크기를 결정한다.

3.4 체크포인트를 캐시할 것인가, 독립 압축기를 둘 것인가

식 (4)에는 설계 선택이 하나 있다 — **하나의 메모리가 이어 달리며 남긴 체크포인트를 캐시할 것인가, 아니면 세그먼트마다 독립메모리 모듈로 압축할 것인가.**

1. **최적화 관점**: 연상 기억을 훈련시키는 과정으로 보면, token 은 훈련 샘플이고 과거 지식의 망각을 피하려 **훈련 (최적화) 과정의 체크포인트를 캐시**한다. 이때 각 세그먼트의 메모리는 이전 세그먼트의 마지막 상태에서 시작한다: $M_0^{(s)} = M_{L^{(s-1)}}^{(s-1)}$.
2. **압축 관점**: 캐시가 그 세그먼트 정보의 **압축된 대표**이길 원한다면, 세그먼트 간 간섭을 피하기 위해 **독립 초기값**에서 시작하는 독립 메모리를 쓴다.

실전에서는 두 선택 각각에 장단점이 있음을 관찰한다 (5.6 절).

4 논의와 개념 증명 (Discussion and Proof of Concept)

4.1 linear/deep memory 에 대한 함의

Linear memory 극단. 세그먼트 크기 1, 벡터값 value-less 메모리라는 극단을 보자. 각 token 이 하나의 세그먼트가 되고 메모리는 그 token 만 저장한다: $M_1^{(t)} = b_K + x_t W_K (M_0^{(t)})$ 은 bias 역할). 여기에 MC 를 적용하면

$$y_t = \sum_{i=1}^t \gamma_t^{(i)} M_1^{(i)}(q_t) = \sum_{i=1}^t \frac{\exp(u_t^\top k_i)}{\sum_{\ell} \exp(u_t^\top k_\ell)} (b_K + x_i W_K) q_t \quad (19)$$

— 재파라미터화하면 **softmax attention** 과 닮은 형태가 복원된다. 즉 MC 는 “N 을 키우면 attention 에 다가가고, N 을 줄이면 RNN 으로 돌아오는” 스펙트럼 위에 있다.

4.2 세그먼트 분할이용량 · 복잡도에 미치는 효과

세그먼트 길이 배열이 유효 용량과 비용을 함께 결정한다. **균일분할** (모든 세그먼트 같은 길이) 과 **로그 분할** (최근은 짧게, 먼과거는 굵게) 등의 스킴을 논한다 — 로그 분할은 캐시 수를 $O(\log L)$ 로 늘려비용을 줄이면서 최근 정보의 해상도를 지킨다 (그림 3).

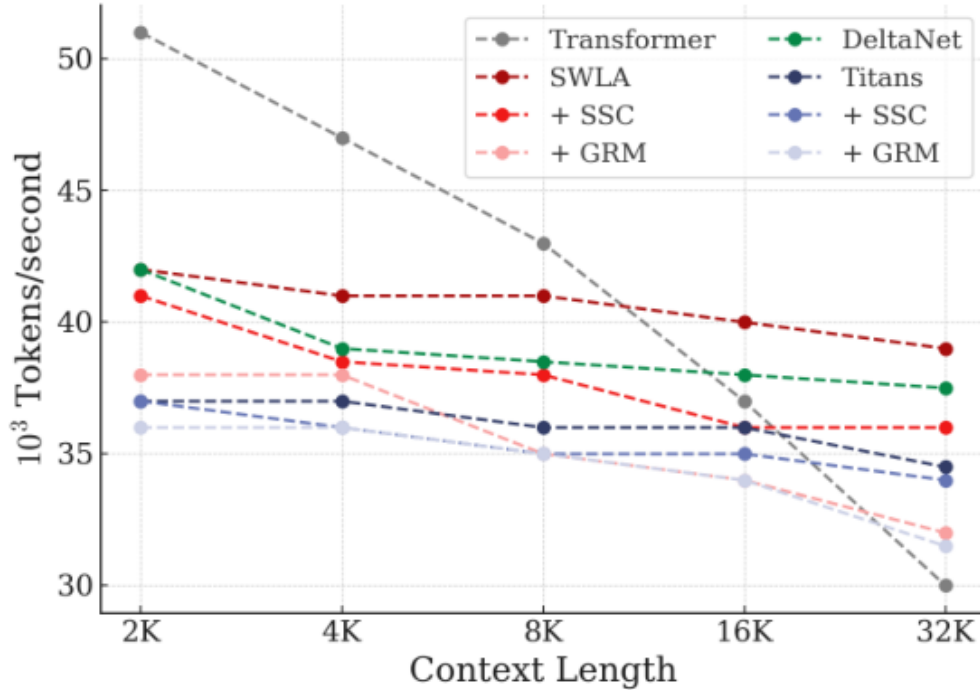


Figure 4: Training throughput comparison

Transformers, and performs better than state-of-the-art to larger memory capacity that scales with sequence length.

Table 4: Accuracy on LongBench tasks (Bai et al., 2024): MultiQA, Musique, GovReport, QMSum, MultiNews, TREC

Model	Single-Doc QA			Multi-Doc QA	
	NQA	QQA	MFQ	HQA	2WM
Transformer	11.5	9.6	19.1	21.5	28.9
DLA	9.4	17.5	12.1	11.8	22.3
+ Log-Linear++	10.1	10.2	17.1	12.4	23.3
+ GRM	11.6	10.3	19.8	18.2	26.9
+ Memory Soup	11.2	10.3	19.5	16.7	25.1
+ SSC	10.7	10.2	18.8	14.2	24.8
Titans (LMM)	8.7	12.5	18.4	15.6	26.1
+ Log-Linear++	9.6	8.9	19.3	18.7	26.9
+ GRM	11.8	9.4	19.9	21.4	29.1
+ Memory Soup	10.7	9.2	19.6	20.2	28.2

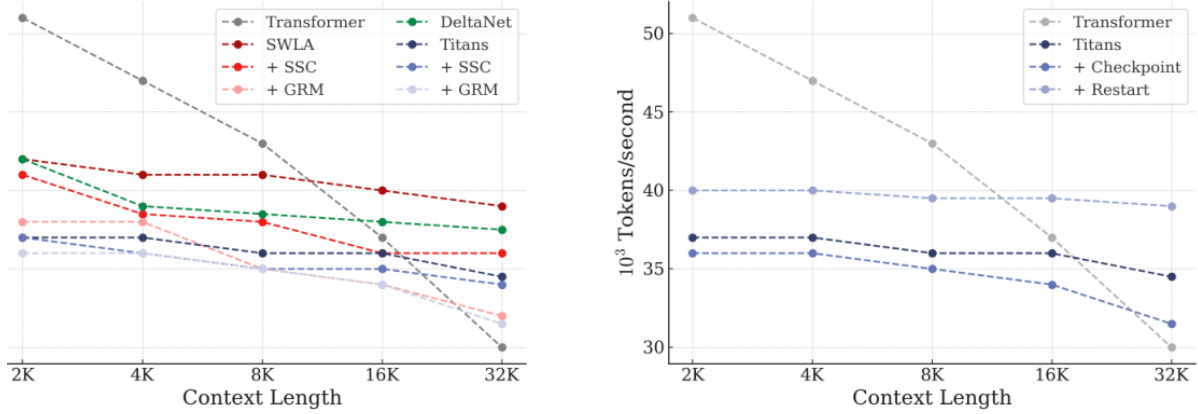


Figure 4: Training throughput comparison of memory caching variants and baselines.

그림 4: MC 변형과 baseline 의 훈련 처리량 비교. (출처: 원저자, arXiv:2602.24281 Fig.4)

5.4 긴 문맥 이해 (LongBench)

장문 이해 과제에서도 MC 가 base 대비 개선을 보인다.

5.6 Ablation

- γ 가 입력만의 함수 vs 입력+블록 문맥의 함수: 문맥 결합 (식 10) 이 평균적으로 유의미하게 낮다.
- 게이트 제거 (= residual memory 로 붕괴): 이 단순 설계조차성능을 높인다.
- 선형 메모리 사용: 놀랍게도 MC 를 쓰면 메모리아키텍처 · 표현력에 대한 성능의 강건성이 커진다.

5.7 효율

훈련 처리량에서 MC 변형들은 **Transformer** 와 **RNN** 사이의 **중간지대**를 제공하며, 문맥이 길어 질수록 Transformer 대비 극도로 효율적이된다. **SSC** 가 양쪽의 장점을 모두 갖는다 — 다양한 downstream 에서다른 변형과 대등하거나 더 나으면서, base RNN 대비 오버헤드는 최소이고 긴 시퀀스에서 효율이 특히 좋다 (그림 4).

6 결론 (Conclusion)

우리는 모든 RNN 에 적용 가능한 단순 기법 **Memory Caching(MC)** 을제시했다. 메모리 상태의 부분집합을 캐시해, 이후 token 들이 관련 있는과거에 직접 attend 할 수 있게 한다. 실험은 일부 baseline 대비 개선을보였다. 본 논문의 많은 선택은 모델을 최대한 단순하게 유지해 **memory caching** 아이디어 자체의 효과를 잘 드러내기 위한것이였다. 향후에는 더 표현력 있는 pooling·routing 메커니즘으로 성능을 더 끌어올릴 수 있을 것이다.